# On the Hardness of Proving a ZX-Diagram Equality

Mathis Hage, *MSc Student at EPFL, Intern at Geneva University*

**Abstract**

After several years of studies, the ZX-calculus language for diagrammatic reasoning on qubits has finally seen emergence of rulesets ensuring completeness for all sorts of fragments of quantum mechanics. Even though we therefore know that we can always prove or disprove an equality between two ZX-diagrams using only the rules of ZX-calculus, an interesting question could be to ask what is the number of rules necessary to do so. This paper exhibits the difficulty of this question, lying mostly in proof complexity theory. Despite this, after some work to prove NP-membership, I still present serious arguments for NP-hardness of the problem when constrained to rational phases. An extension is then made to the real phases case in the context of the Blum-Shub-Smale Model, before ending by a hypothesis regarding the difficulty of the problem constrained to the Clifford fragment.

**Index Terms**

Quantum Computing, ZX-Calculus, Proof Complexity, NP-hardness, NP, Reduction, Circuit Obfuscation, Blum-Shub-Smale Model.

## I. Introduction

In the last decades, *the ZX-Calculus* language for diagrammatic reasoning on qubits has received a lot of attention for the possibilities it offers in quantum computing. For example, one can use this language to reduce the T-gate count in a quantum circuit. Over the years, the theory behind the ZX-Calculus was hence refined, up to achieving a *complete* form of the langugage [1].

However, a lot of important matters linked to the ZX-Calculus are hard. The most famous one is the circuit extraction problem, i.e. the problem consisting of extracting a quantum circuit from a ZX-Diagram. Even though some efficient algorithms exist in some specific cases, for example with diagrams presenting mathematical structures called *flows* (see for example [2]), circuit extraction remains #P-complete in the general case [3].

In this case study, we look at another aspect of the ZX-Calculus. We said earlier that this language is *complete* : this means that two diagrams are equal if and only if there is a sequence of rules (allowed by the language) transforming one diagram into the other. Therefore it is always possible to go from one diagram to another if they represent the same linear map on qubits. An interesting question can now be asked : what is the *number of steps* needed to prove an equality between two diagrams ? More specifically, let the language :

$$\text{ZX-REWRITE} := \{(D_1, D_2, k) \mid D_1, D_2 \text{ are ZX-diagrams s.t. the equality } D_1 = D_2 \text{ can be proven true in at most } k \text{ steps}\}.$$

What is its complexity class ? This problem not only stems from ZX-calculus theory, but also from a general proof complexity setting. We note that $k$ is written in unary.

## II. Requirements

### A. ZX-Calculus

This part is largely based on *ZX-calculus for the working quantum computer scientist* p. 1-34. [4]. We avoid redundant citation in the rest of this section.

ZX-calculus is a graphical way to represent and reason about linear maps between qubits. A ZX-diagram represents qubits flowing through wires, hitting some nodes, which are linear maps acting on them, and at some point coming out. Such a diagram is composed of multiple instances of a specific set of *generators*.

*1) Generators:* The first generators are the *spiders*. These are nodes that can be green or red, and take a parameter $\alpha$. As in [4], our figures use grey spiders instead of red ones, and white spiders instead of green ones. A green (white) node with $n$ input wires (i.e. connected to the left of the node), $m$ output wires (i.e. connected to the right of the node) and parameter $\alpha$, as shown in fig. (1),

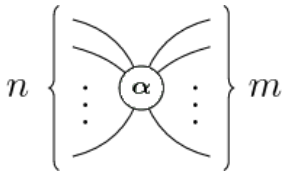


Fig. 1: Green Spider with $n$ inputs, $m$ outputs, and parameter $\alpha$ (taken from [5])

corresponds to the linear map :

$$\overbrace{|0\cdots0\rangle}^{m}\overbrace{\langle 0\cdots0|}^{n} + e^{i\alpha}\overbrace{|1\cdots1\rangle}^{m}\overbrace{\langle 1\cdots1|}^{n}.$$

If the node is red (grey), it corresponds to the linear map :

$$\overbrace{|+\cdots+\rangle}^{m}\overbrace{\langle +\cdots+|}^{n} + e^{i\alpha}\overbrace{|-\cdots-\rangle}^{m}\overbrace{\langle -\cdots-|}^{n}.$$

Here are a few examples :

- A green spider with 0 inputs and 1 output : $|0\rangle + e^{i\alpha}|1\rangle$ ;
- The same spider but red and parameter $\alpha = 0$ corresponds to $|0\rangle$, while with $\alpha = \pi$ it corresponds to $|1\rangle$.
- A green spider with one input and one output corresponds to the rotation matrix (on the Bloch sphere) by an angle $\alpha$ around the $Z$ axis :

$$R_Z(\alpha) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\alpha} \end{pmatrix}$$

There are also other generators. First, the SWAP gate in fig. (2) (from quantum computing theory), simply represented as crossing wires.

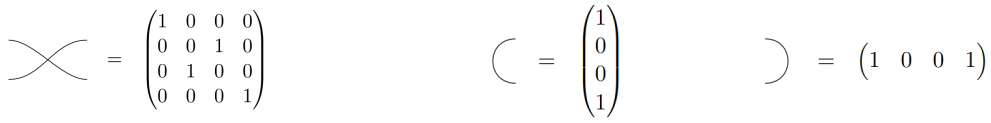

Fig. 2: The SWAP (a) and the cup and cap (b) generators.

One can freely slide spiders along these crossing wires. It is clear from this visual representation that SWAP is self-inverse : try composing two SWAP's side by side, you will notice that both qubits end up in their original lane.

Moreover, two other generators are of fundamental importance, namely the *cup* and the *cap* (respectively) drawn in fig. (2). Notice how they represent the Bell state and the Bell effect. Here also, a spider can freely move on a cup or a cap.

*2) Only connectivity matters:* Because of the fact that the spiders are symmetric, and using the SWAP, cups and caps generators, we arrive at the point that makes ZX-calculus very interesting : this can be summarized by the sentence *"only connectivity matters"*. In fact, we can bend wires in any way we like, see them either as inputs or outputs of spiders, this doesn't change the overall linear map represented by the ZX-diagram as long as we dont change the order of the inputs and outputs of the whole diagram.

ZX-calculus is universal, meaning it can represent any linear map.

*3) Rules:* When working with ZX-diagrams, and in particular when trying to simplify them, one can make use of a set of rules to do so. These rules are summarized in fig. (3). They do not change the linear map when used.

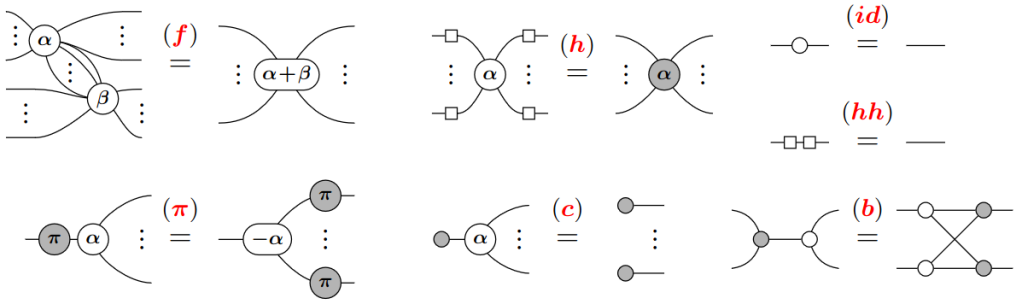


Fig. 3: The rules of ZX-calculus making it complete for the Clifford fragment, namely : *spider fusion* ($f$), *hadamard* ($h$, $hh$), *identity* ($id$), *commute* ($\pi$), *copy* ($c$), *bialgebra* ($b$). Note that the square box represents the hadamard unitary, which can be built from spiders as well.

There also is a set of practical "meta"-rules, which can be derived from the above ones. While we do not detail them all, we simply mention that the rules of fig. (3) all hold with the colors interchanged, and with the inputs / outputs interchanged.

*4) Completeness:* The ZX-Calculus is *complete*, meaning we can prove or disprove any equality between diagrams by using only the rules of the language. The rules of fig. (3) makes it complete for the *Clifford fragment*, meaning when the phases are multiples of $\pi/2$. For completeness when the phases are arbitrary, we use the rules described in [1], page 34, fig. 6.

Note that we work with unnormalized quantum states, which is not an issue.

## III. Arbitrary rational phases

Now that the prerequisites have been introduced, we can dive into the heart of the problem. Intuitively, even by knowing that two diagrams represent the same linear map, the problem of finding a sequence of ZX-rules of length at most $k$ to go from one to the other seems like a very difficult task. Even though a verifier running in polytime can be found relatively easily, hence proving NP-membership of the language ZX-REWRITE, it is much more complicated to prove the suspected NP-hardness. In fact, unlike problems that require checking if two diagrams are equal for example, we have to focus on the *topology* of the diagrams rather than on the transformation they represent.

To be able to conveniently work in the standard Turing machines setting, we constrain ourselves to having only rational phases, as for them to have a finite presentation to pass the verifier.

### A. NP-*membership*

**Theorem 1.** ZX-REWRITE *is in* NP.

*Proof.* We claim that alg. (1) constitutes a valid polytime verifier for ZX-REWRITE. In fact, this algorithm simply applies the rules described by the certificate (there should be less than $k$ of them) to the first diagram, and accepts if we recover the second one exactly. A few remarks to ensure that this works :

- Phases are rational : they can perfectly be passed as inputs.
- No rule makes the number of nodes explode exponentially.
- We can easily adopt a naming convention for the spiders and the wires as well as a convention on how to change the labels when using a rule. For example, each time a spider is created, we can choose to set its label to be the max of the existing labels $+1$. This way the certificate can coherently describe how to use each rule, and be understood by the verifier.

**Algorithm 1** Verifier $V$

**Input:** $\langle D_1, D_2, k, C\rangle$

Parse $C$ as $\langle R, L\rangle = \langle (r_1, \dots, r_n), (l_1, \dots, l_n)\rangle$ where $r_i$ is a rule to apply, and $l_i$ contains the details on how to apply it: for example for the fusion rule, $l_i$ would contain the two vertices to fuse, and for the other direction (un-fusion), it would contain the spider to unfuse, how to split its phase, and how to spread the wires among the two new spiders.
**if** $n > k$ **then**
    Reject.
**end if**
Otherwise, $D \leftarrow D_1$
**for** $i = 1, \dots, n$ **do**
    $D \leftarrow D$ with rule $r_i$ applied
**end for**
Output **true** if and only if $D = D_2$.

□

### B. NP-*hardness*

In this proof tentative, we conjecture that the hardness of the problem partially comes from the fact that the phases of the spiders can be arbitrary. In fact, it seems very difficult for an algorithm to specifically use the "unfusion" rule to split a phase $\gamma \in \{x \in \mathbb{Q} \mid 0 \leq x < 2\pi\}$ into $\gamma = \alpha + \beta$.

Define the fusion-rule-only version of the language :

$$\text{ZX-REWRITE}_F := \{\langle D_1, D_2, k\rangle \in \text{ZX-REWRITE} \mid \text{we can go from } D_1 \text{ to } D_2 \text{ using only the fusion rule at most } k \text{ times}\}.$$

We can in fact prove NP-hardness of this language by a reduction from SUBSET-SUM. We start by reminding its definition.

**Definition 2.** SUBSET-SUM

$$\text{SUBSET-SUM} := \{(S, t) \mid S \subset \mathbb{N} \textit{ is finite, } t \in \mathbb{N}\textit{, and } \exists\, U \subseteq S : \sum_{s \in U} s = t\}.$$

**Algorithm 2** Reduction from SUBSET-SUM to ZX-REWRITE$_F$

**function** $f(\langle S, t\rangle)$
  Let $n = |S|$
  **if** $t > \sum_{s\in S} s$ **then**
    Let $D_1$ be a single 0-phase green spider with one input and one output.
    Let $D_2$ be a wire containing two 0-phase green spiders.
    **return** $\langle D_1, D_2, 0\rangle$
  **else**
    $\mathcal{N} \leftarrow \sum_{s\in S} s$
    Create $D_1$ and $D_2$ as shown in fig. (4).
    **return** $\langle D_1, D_2, n\rangle$
  **end if**
**end function**

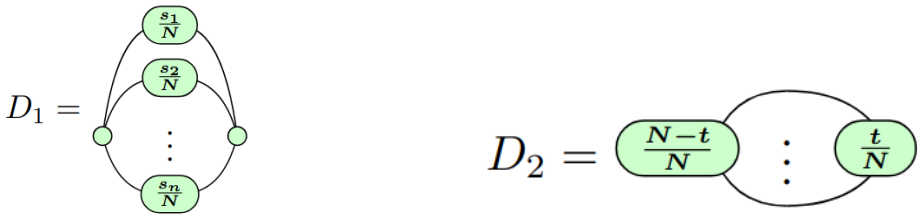


Fig. 4: Diagrams created by the reduction in alg. (2). The three dots in $D_2$ indicate $n$ wires.

**Theorem 3.** SUBSET-SUM $\leq_p$ ZX-REWRITE$_F$.

*Proof.* Consider the function $f$ : SUBSET-SUM $\rightarrow$ ZX-REWRITE$_F$ computed by alg. (2).

Clearly, this algorithm runs in time $\mathcal{O}(n)$ where $n := |S|$, and hence $f$ is a polynomial time computable function. Now, we need to show :

$$\langle S, t\rangle \in \text{SUBSET-SUM} \Leftrightarrow f(\langle S, t\rangle) = \langle D_1, D_2, k\rangle \in \text{ZX-REWRITE}_F.$$

Clearly if $t > \sum_{s\in S} s$, the reduction outputs a string that is not in ZX-REWRITE$_F$, as it is impossible to go from $D_1$ to $D_2$ in $k = 0$ steps (they are not equal).

Otherwise, $k = n$ and :

($\Rightarrow$) $\langle S, t\rangle \in$ SUBSET-SUM $\implies$ $\exists\, U \subseteq S$ s.t. $\sum_{s\in U} s = t$. Start with $D_1$. By fusing every spider of the form $\frac{s}{\mathcal{N}}, s \in U$ with the right 0-phase spider, its phase becomes $\sum_{s\in U} \frac{s}{\mathcal{N}} = \frac{1}{\mathcal{N}} \sum_{s\in U} s = \frac{t}{\mathcal{N}}$. By fusing the remaining ones with the left 0-phase spider, its phase becomes $\sum_{s\notin U} s = \frac{\mathcal{N}-t}{\mathcal{N}}$. Hence we have recovered $D_2$, in $n$ steps exactly.

($\Leftarrow$) It is possible to go from $D_1$ to $D_2$ in $n$ steps. Consider such a sequence of $n$ steps using only the fusion rule. Hence we necessarily had to fuse the $n$ center spiders one by one to the left or to the right. By creating a set $U$ containing all $s \in S$ such that node $\frac{s_i}{\mathcal{N}}$ was fused to the right, we have that $\langle S, t\rangle \in$ SUBSET-SUM.

Therefore, $f$ is a valid reduction, which proves SUBSET-SUM $\leq_p$ ZX-REWRITE$_F$.

□

**Corollary 4.** ZX-REWRITE$_F$ *is* NP*-hard.*

This however does not prove NP-hardness of the original ZX-REWRITE. In fact, the original problem contains much more rules to modify a diagram, which, in the second part of the proof, could allow to go from $D_1$ to $D_2$ without using only the fusion rule. Therefore, we must argue that a valid sequence of $n$ steps may only contain the fusion rule to be able to go from $D_1$ to $D_2$, or find a counterexample, which would invalidate the reduction.

We argue the former : since we have to go from $n + 2$ to 2 spiders, we have to use (at some point) a rule reducing the number of spiders. However, taking a look at the ruleset [1] page 34, fig 6., we see that no rule other than spider fusion can get us closer to $D_2$ when applied. Hence we need to have $n$ fusion steps, as desired.

**Claim 5.** ZX-REWRITE *is* NP*-hard.*

### *C. Comment on the introduction of real phases*

This part was based on a course by Olivier Bournez [6].

When dealing with real numbers, one can work in the context of Blum-Shub-Smale (BSS) machines [7]. A BSS-machine is a computation model that is analogous to a Turing machine but designed specifically to work with real numbers. Typically, the tape of 0's and 1's is replaced by a tape (infinite vector) of real numbers. Then, the machine can perform different types of operations on elements of the tape.

Let us limit ourselves to *linear* BSS-machines, i.e. that can only perform actions among $(.,+,-,=)$ where "." denotes mulitplication by a scalar, and " $=$ " means testing if some value is equal to zero. In this case, we get the desirable result that when considering only boolean inputs, a linear BSS-machine is equivalent to a Turing machine, in the sense that :

$$\mathrm{P} = \mathrm{BP}(\mathrm{P}^{=}_{\mathrm{lin}}),$$
$$\mathrm{NP} = \mathrm{BP}(\mathrm{NP}^{=}_{\mathrm{lin}}).$$

Here, we denote complexity classes of linear BSS-machines with the superscript " $=$ " and the subscript "lin". Moreover, $\mathrm{BP}(\mathcal{C})$ denotes the *Boolean Part* of a complexity class $\mathcal{C}$ :

$$\mathrm{BP}(\mathcal{C}) := \{L \cap \{0,1\}^* \mid L \in \mathcal{C}\}.$$

In this setting, SUBSET-SUM (called KNAPSACK in [6]) is still also $\mathrm{NP}^{=}_{\mathrm{lin}}$-hard, and we could analogously define a reduction from it to our ZX-REWRITE problem.

We note that to this day, the interpretation of this result is different, in the sense that it is already known that :

$$\mathrm{P}^{=}_{\mathrm{lin}} \neq \mathrm{NP}^{=}_{\mathrm{lin}}. \quad [6]$$

## IV. Clifford Fragment

We now limit ourselves to the *Clifford circuits*, i.e. the diagram will contain only phases that are multiples of $\pi/2$, and will have as many outputs as they have inputs. Intuitively, the problem seems at most as hard as the case with only rational phases : not only is the number of possible phases now discrete ($\alpha \in \{0, \pi/2, \pi, 3\pi/2\}$), but the set of rules is also reduced (the rules described in fig. (3) now suffice). Let ZX-REWRITE$_{CC}$ be the problem constrained to ZX-diagrams originating from Clifford circuits.

It is not too hard to see that the problem is in NP : with a proper labelling of the possible phases, alg. (1) still works as a polytime verifier for ZX-REWRITE$_{CC}$. The more interesting question would be to know if this problem could be in P. It turns out this question is still very complicated : after careful thinking, it seems unlikely that a polytime algorithm could be found easily to decide this problem (unless P = NP): how could we know that a valid sequence of steps is minimal without enumerating them all ? In the meantime, one still needs to find a different reduction than the one used previously to prove NP-hardness. Unfortunately due to a lack of time, this has not been achieved, and no polytime verifier has been identified. We nevertheless state our hypothesis :

**Hypothesis 6.** ZX-REWRITE *is* NP*-hard.*

## V. Conclusion

To conclude this study, the matter of proving a ZX-diagram equivalence seems like a very hard problem. While membership to the NP class (or equivalent for real phases) can be shown after some work, NP-hardness remains much harder to prove. Even though I was able to exploit the difficulty of deciding phase splits to produce an arguably coherent reduction, the complexity of some of the ZX-rules and their large number makes it hard to formally prove that the reduction is correct. Clearly, adding the constraint of proof length $k$ to the problem of going from $D_1$ to $D_2$ causes a complete change of field : we go from having to prove that a path exist between two diagrams representing the same map, i.e. completeness, which has already been extensively studied, to having to prove in how much steps this can be done, which lies almost completely in general proof theory (and has not been covered so much in the context of ZX-calculus). An interesting point to focus on would be finding a solution to the restriction of my problem to the Clifford fragment : one could pinpoint where does the hardness of the problem could come from, and construct a reduction accordingly. Conversely, as this has been done numerous times for other aspects of the ZX-calculus such as circuit extraction, it would be unlikely but not too surprising to find a polytime algorithm solving the problem, invalidating my hypothesis.

## Acknowledgment

A special thanks to Matteo De Francesco, PhD. candidate at the university of Geneva, for supervising me during my internship, and coming up with the idea of the research problem. I am also very grateful to Professor Arnaud Casteigts, head of the ALGO lab at the university of Geneva, for accepting to have me as an intern. A final thank you to both of them as well as the whole team for all the priceless advice they gave me and the nice chats we had as well.

## REFERENCES


[1] E. Jeandel, S. Perdrix, and R. Vilmart, "Completeness of the zx-calculus," *Logical Methods in Computer Science*, vol. Volume 16, Issue 2, 2020. [Online]. Available: http://dx.doi.org/10.23638/LMCS-16(2:11)2020

[2] A. Kissinger and J. van de Wetering, "Zx-flow: A flexible criterion for deterministic computation with zx-diagrams," 2026. [Online]. Available: https://arxiv.org/abs/2603.09580

[3] M. Backens, H. Miller-Bakewell, G. de Felice, L. Lobski, and J. van de Wetering, "There and back again: A circuit extraction tale," *Quantum*, vol. 5, p. 421, Mar. 2021. [Online]. Available: http://dx.doi.org/10.22331/q-2021-03-25-421

[4] J. van de Wetering, "Zx-calculus for the working quantum computer scientist," 2020. [Online]. Available: https://arxiv.org/abs/2012.13966

[5] Wikipedia contributors, "Zx-calculus," https://en.wikipedia.org/w/index.php?title=ZX-calculus, 2026, [Online; accessed August 10, 2026].

[6] O. Bournez, "Course notes for mpri course 2.33.1: Blum Shub Smale model," February 2013, available at https://wikimpri.dptinfo.ens-paris-saclay.fr/lib/exe/fetch.php?media=cours:upload:bss.pdf.

[7] Wikipedia contributors, "Blum–shub–smale machine," https://en.wikipedia.org/wiki/Blum-Shub-Smale_machine, 2026, [Online; accessed August 14, 2026].